GOOGLE | 1POINT3ACRES | TOPIC 1 OF 10 | COMMON TOPIC

# Arrays, Hashing and Two Pointers

15 questions to study here, each on its own page, with the full interview answer and code

14 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

| 2 | 12 | 1 |
|---|---|---|
| HARD | MEDIUM | EASY |

**How to use**

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

**Go to Contents >**

# Contents

**Arrays, Hashing and Two Pointers | 15 questions | tap a title to jump to it**


| | | |
|---|---|---|
| **H** | 1. Maximum-Area Rectangle from Planar Points | **2026-09-29** |
| **H** | 2. Necklace Cut into Two Halves with Equal D/R Counts | **2026-05-03** |
| **M** | 3. Connected Crop Arrangement on a Grid | **2026-10-04** |
| **M** | 4. Range Duplicate Check with Constant-Time Queries | **2026-09-29** |
| **M** | 5. Temperature Monitoring Data Structure | **2026-09-10** |
| **M** | 6. Longest Same-Color Grid Sequence | **2026-09-08** |
| **M** | 7. Next-Word Predictor | **2026-09-02** |
| **M** | 8. Piano Hand Movement Grouping | **2026-07-14** |
| **M** | 9. Website Activity Analytics: Unique Users + Session Time | **2026-06-26** |
| **M** | 10. Gold Chain Split into Two Equal Halves | **2026-04-21** |
| **M** | 11. Sum of Good Subarrays (Adjacent Diff ±1) | **2026-02-08** |
| **M** | 12. Run-Length Vector Storage + Dot Product | **2026-01-21** |
| **M** | 13. Two-Day Log Intersection under Memory Limit | **2025-11-27** |
| **M** | 14. Rotate Any m×n 2D Matrix | |
| **E** | 15. Robot Status Message Deduplication Window | **2026-09-10** |


**SKIP: these fit this topic but have their pages in another topic (14)**


| | | | |
|---|---|---|---|
| **M** | SKIP | Visible People in a Line (Last Person, Then Everyone) | **topic 4: Heaps, Stacks and Queues** |
| **M** | SKIP | Longest Non-Decreasing Subarray with One Replacement | **topic 2: Dynamic Programming** |
| **M** | SKIP | Meeting Rooms / Interval Scheduling Variants | **topic 4: Heaps, Stacks and Queues** |
| **M** | SKIP | Active Communication Top Users | **topic 4: Heaps, Stacks and Queues** |
| **M** | SKIP | Union of Sorted Interval Lists | **topic 4: Heaps, Stacks and Queues** |
| **M** | SKIP | Cutting Ribbons (LC 1891) | **topic 6: Binary Search and Sorting** |
| **M** | SKIP | Target Expressions from Ordered Digits | **topic 2: Dynamic Programming** |
| **M** | SKIP | Progressive Array Partitioning | **topic 6: Binary Search and Sorting** |
| **M** | SKIP | Top-K Frequent in Large Logs | **topic 4: Heaps, Stacks and Queues** |
| **M** | SKIP | Streaming Points: Emit K Within Distance | **topic 6: Binary Search and Sorting** |
| **M** | SKIP | Greedy Longest-Match Dictionary Tokenizer | **topic 5: Trees and Tries** |
| **M** | SKIP | Employee Hierarchy Add / Delete | **topic 5: Trees and Tries** |
| **M** | SKIP | Jump Game with Profit | **topic 2: Dynamic Programming** |
| **M** | SKIP | Russian Doll Envelopes | **topic 2: Dynamic Programming** |

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Greedy and Math (listed there as SKIP, no pages). Site tags: math, hashmap, hard

**HARD | 1Point3Acres | onsite-coding | 1 of 15**

# Maximum-Area Rectangle from Planar Points

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 177/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #963 Minimum Area Rectangle II

**LeetCode match:** LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

## QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

**Input:** a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

## APPROACH

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

## HOW TO THINK ABOUT IT

Four points form a rectangle exactly when two of its diagonals share the same midpoint and have the same length. So you never need to pick four points: bucket every pair of points by (sum of the two points, squared distance). Two different pairs in one bucket are the two diagonals of a rectangle, and the area is the cross product of two adjacent sides.

## HOW TO THINK ABOUT IT

**Basis:** LeetCode 963, Minimum Area Free Rectangle, with the objective flipped to maximum area.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

**WHY IT IS SLOW**

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

## OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

| TIME | SPACE |
|---|---|
| O(n^2) pairs (worst case O(n^3) if every pair shares a bucket) | O(n^2) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict
from itertools import combinations


def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
```

```
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx


# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)


# BRUTE FORCE (say it first)
def max_rect_area_cubic(points):
    # first idea to say aloud: try every corner triple
```

```
    pts = set(map(tuple, points))
    best = 0
    for p in pts:
        for q in pts:
            for r in pts:
                if len({p, q, r}) < 3:
                    continue
                # right angle at p between p->q and p->r
                dot = ((q[0] - p[0]) * (r[0] - p[0])
                       + (q[1] - p[1]) * (r[1] - p[1]))
                # the fourth corner must also exist
                s = (q[0] + r[0] - p[0],
                     q[1] + r[1] - p[1])
                if dot == 0 and s in pts:
                    area = abs(cross(p, q, r))
                    best = max(best, area)
    return best


if __name__ == "__main__":
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    assert max_rect_area_cubic(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6
```

## NOTES FROM CANDIDATES

Candidates describe it as LC 963 (Minimum Area Rectangle II) with the objective flipped to maximum area.
Asked as a standalone L4 onsite coding round; in another L4 onsite it was the second problem after a CPU scheduling question, and the interviewer explicitly expected a better-than-cubic solution.
Clarify up front: whether duplicate points can appear, what to return when no rectangle exists (commonly 0), and whether integer or floating-point coordinates are used (area precision matters for rotated rectangles).

## PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Strings, Parsing and Recursion, Greedy and Math (listed there as SKIP, no pages). Site tags: string, two-pointer, math-reasoning, hard

**HARD | 1Point3Acres | onsite-coding | 2 of 15**

# Necklace Cut into Two Halves with Equal D/R Counts

**ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 188/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

**Original:** https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

## QUESTION

**EXAMPLES**

```
s = "DRDR"    -> cut between index 1 and 2: "DR" +
    "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR");
    rotate, two cuts at offsets 1 and 3:
              pieces "DR" and "DR".
```

## APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

**BRUTE FORCE TO SAY OUT LOUD**

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

**OPTIMAL IDEA**

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

**DRY-RUN THESE CASES**

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

**LIKELY FOLLOW-UPS**

Equal strings instead of equal counts, or three pieces.

**PITFALLS**

## INTERVIEW-DAY PLAN

Assuming a linear cut is always enough. Ignoring parity.

## HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

**Basis:** Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

**WHY IT IS SLOW**

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

## OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

**FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?**

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

**FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.**

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

**FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.**

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate ...
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None
```

```
# FOLLOW-UP 2, 3, 4: What if the two pieces must be equal
# strings, not just equal counts?
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


if __name__ == "__main__":
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None
```

**NOTES FROM CANDIDATES**

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.
Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.
Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.
Verbalize the 2-cut bound out loud before coding — interviewers ask why.

**PREPARATION**

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Grids and Simulation, Greedy and Math (listed there as SKIP, no pages).
Site tags: grid, array, greedy, medium

**MEDIUM | 1Point3Acres | phone-screen | 3 of 15**

# Connected Crop Arrangement on a Grid

**ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 207/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

## QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other. Return any valid arrangement.

**Follow-up:** two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

**EXAMPLES**

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

**or**

```
11
11
222
222
222
```

## APPROACH

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

## HOW TO THINK ABOUT IT

Walk the grid in a snake (left to right on one row, right to left on the next) so that every cell is next to the one before it. If you give crop 1 the first count[1] cells of that walk, crop 2 the next count[2] cells and so on, each crop is a contiguous stretch of the walk, so its cells are connected. No search or backtracking is needed.

**Basis:** Written for this question: a constructive fill along a Hamiltonian path of the grid (a snake), no LeetCode problem matches it exactly.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

| TIME | SPACE |
|---|---|
| exponential | O(n * m) |

**WHY IT IS SLOW**

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

## OPTIMAL SOLUTION · Fill crops along a snake path

| TIME | SPACE |
|---|---|
| O(n * m) | O(n * m) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# BASE QUESTION (optimal): Fill crops along a snake path
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)


def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
```

```
    return path


# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# M x M).
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)


def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True


if __name__ == "__main__":
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
```

```
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
    assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
               for a, b in zip(p, p[1:]))
```

### NOTES FROM CANDIDATES

Asked as the first-round online coding interview of an early-career L3 loop.

Because any valid arrangement is accepted, look for a constructive traversal order that keeps consecutive cells adjacent, rather than backtracking.

For the follow-up, the two gardens can be stacked side by side or one above the other; clarify how they are joined (which edges touch and the offset) before designing the traversal.

## PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Site tags: array, sliding-window, hashmap, prefix-sum, medium

**MEDIUM | 1Point3Acres | onsite-coding | 4 of 15**

# Range Duplicate Check with Constant-Time Queries

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 211/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

**Original:** https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

## QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

**Base:** given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

## APPROACH

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

## HOW TO THINK ABOUT IT

A range [l, r] has a duplicate exactly when some index i in it has its next equal element at an index <= r. Let next[i] be the next index holding the same value and reach[l] the smallest next[i] over all i >= l. Then the range is all-unique iff reach[l] > r, a single comparison per query after one linear pass.

**Basis:** Written for this question (a precomputed next-occurrence array, similar in spirit to LeetCode 3, Longest Substring Without Repeating Characters).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

| TIME | SPACE |
|---|---|
| O(r - l) per query | O(r - l) |

**WHY IT IS SLOW**

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

## OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

| TIME | SPACE |
|---|---|
| O(n) preprocessing, O(1) per query | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import Counter

INF = float("inf")


# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1


# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
```

```
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# FOLLOW-UP 1: The array changes between queries.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
```

```
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


if __name__ == "__main__":
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
    # an update can create or remove a duplicate
    q.update(3, 9)
    assert q.unique(0, 4)
    q.update(0, 9)
    assert not q.unique(0, 3)
```

**NOTES FROM CANDIDATES**

In one L4 onsite, this was the second question of a round that opened with weighted job scheduling; the interviewer asked specifically for the precomputation that makes each query O(1). The interviewer states the complexity targets explicitly, so the round is judged on hitting them, not on a working brute force. Clarify whether the array can change between queries (one version mentions linear-time updates) and whether ranges are inclusive.

**PREPARATION**

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: data-structure, streaming, sliding-window

**MEDIUM | 1Point3Acres | onsite-coding | 5 of 15**

# Temperature Monitoring Data Structure

**ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 223/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

## QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days. SWEdata-structurestreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

## APPROACH

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether the window is the last N readings or the last N days, what moving average should return before the window fills, and the number of readings.

**BRUTE FORCE TO SAY OUT LOUD**

Store everything in a list and compute the window average and max by scanning. O(window) per query.

**OPTIMAL IDEA**

For a count window keep a deque of the last N readings and a running sum for the average, and a monotonic decreasing deque for the maximum: pop smaller values from the back on insert and drop the front when it expires. All operations O(1) amortized. For time windows store (time, value) and evict by timestamp.

**DRY-RUN THESE CASES**

N = 1, duplicate maxima, negative readings, the maximum expiring, fewer readings than the window.

**LIKELY FOLLOW-UPS**

## INTERVIEW-DAY PLAN

Minimum too, median over a window, or out-of-order timestamps.

**PITFALLS**

Removing the wrong element from the max deque when equal values repeat. Floating-point drift in the running sum.

## HOW TO THINK ABOUT IT

Three operations on the last N readings. The average only needs a running sum, and the maximum needs a monotonic deque that drops smaller values from the back as larger ones arrive, so the front is always the max of the window. Decide first whether the window is a count of readings or a span of time.

**Basis:** Written; a sliding window with an average (LeetCode 346) and a window maximum (LeetCode 239), here as one data structure.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

| TIME | SPACE |
|---|---|
| O(N) per average or max | O(N) |

**WHY IT IS SLOW**

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

## OPTIMAL SOLUTION · Running sum plus a monotonic deque

| TIME | SPACE |
|---|---|
| O(1) amortized per operation | O(N) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

**FOLLOW-UP 2 Also support minimum, or the median.**

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

**FOLLOW-UP 3 Floating-point drift in the running sum.**

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import deque


class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# FOLLOW-UP 2: Also support minimum, or the median.
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
```

```
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)


# FOLLOW-UP 1: The window is the last D days (elapsed
# time), not the last N readings.
class TimedMonitor(SlidingStats):
    """Window = readings from the last `span` days."""

    def __init__(self, span):
        super().__init__()
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        # drop readings at or before day - span
        cutoff = day - self.span
        self.expire(lambda key: key <= cutoff)
```

```
if __name__ == "__main__":
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    # window is [1, 4, 2]
    assert m.maximum() == 4 and m.minimum() == 1
    assert m.average() == 7 / 3
    m.insert(0)
    # window is [4, 2, 0]
    assert m.maximum() == 4 and m.minimum() == 0
    t = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        t.insert(day, x)
    # day 1 is two days old, so it left the window
    assert t.maximum() == 5 and t.average() == 4
```

## NOTES FROM CANDIDATES

The introductory description refers to the most recent N days, while the API requirements refer to the most recent N readings. Clarify the window definition before coding; the day-based variant does not specify how timestamps enter the interface. Empty-window behavior and insertion complexity are unspecified.

For the reading-count window with positive N, retain the last N readings in a FIFO queue and maintain their sum. Add the new reading and subtract any expired reading. Agree whether a partially filled window returns its current mean or is unavailable.

Maintain a second deque of reading indices with nonincreasing temperatures. Expire old indices at the front and remove smaller or equal temperatures from the back before adding the new index. The front holds the current maximum; indices distinguish duplicate readings.

Average and maximum queries are O(1). Each reading enters and leaves each deque at most once, so insertion is amortized O(1) but can take O(N) for one update; auxiliary space is O(N).

This implements the reading-count contract. A day-based contract additionally needs timestamps, expiration timing, and an ordering rule.

## PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, grid

**MEDIUM | 1Point3Acres | phone-screen | 6 of 15**

# Longest Same-Color Grid Sequence

**ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS**

**STUDY: TALK** **ALL TIME 19.6** **30 DAYS 0** **PRIORITY 148/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

## QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

**EXAMPLES**

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

## APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

## HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

**Basis:** LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

| TIME | SPACE |
|---|---|
| O(m * n * max(m, n)) | O(1) |

**WHY IT IS SLOW**

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

## OPTIMAL SOLUTION · Four run lengths per cell

| TIME | SPACE |
|---|---|
| O(m * n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Four run lengths per cell
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
```

```
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
```

```
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

## NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

## PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.

Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Strings, Parsing and Recursion, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: string, hashmap, data-structure

**MEDIUM | 1Point3Acres | onsite-coding | 7 of 15**

# Next-Word Predictor

**ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 224/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

## QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

## APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.

Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

**BRUTE FORCE TO SAY OUT LOUD**

Store all sentences and scan them for each prediction. O(corpus) per query.

**OPTIMAL IDEA**

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

**DRY-RUN THESE CASES**

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

**LIKELY FOLLOW-UPS**

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

## INTERVIEW-DAY PLAN

**PITFALLS**

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

## HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

**Basis:** Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

| TIME | SPACE |
|---|---|
| **O(corpus size)** per prediction | **O(corpus size)** |

**WHY IT IS SLOW**

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

## OPTIMAL SOLUTION · Counts plus a maintained best successor

| TIME | SPACE |
|---|---|
| **O(1)** per pair when training and per prediction | **O(number of distinct pairs)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Return the top K predictions.**

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

**FOLLOW-UP 2 Use two words of context (trigrams).**

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

**FOLLOW-UP 3 The corpus keeps changing and old text should fade.**

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# FOLLOW-UP 1, 2: Return the top K predictions.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None


    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"
```

**NOTES FROM CANDIDATES**

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.

The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.

After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.

With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.

Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

**PREPARATION**

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, simulation, sliding-window, medium

**MEDIUM | 1Point3Acres | onsite-coding | 8 of 15**

# Piano Hand Movement Grouping

**ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 205/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

## QUESTION

**EXAMPLES**

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the
    stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] ->
    1, without stating the reach parameter.
```

## APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

**BRUTE FORCE TO SAY OUT LOUD**

Try every starting position for each group and take the minimum. Exponential or quadratic.

**OPTIMAL IDEA**

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

**DRY-RUN THESE CASES**

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

**LIKELY FOLLOW-UPS**

A hand that can slide, two hands, or return the group boundaries.

## INTERVIEW-DAY PLAN

**PITFALLS**

Counting the initial placement inconsistently. Assuming notes are sorted.

## HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

**Basis:** Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

## OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

**FOLLOW-UP 2 Count lifts, not placements.**

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

**FOLLOW-UP 3 The hand may slide without a lift (costing something).**

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))


# FOLLOW-UP 2: Count lifts, not placements.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)


# FOLLOW-UP 1: Return where each group starts and where to
# place the hand.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out


if __name__ == "__main__":
    # the stated example: 2 placements for reach 4
```

```
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2
```

**NOTES FROM CANDIDATES**

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.
The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.
Maintain the feasible placement interval for the current group.
The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

**PREPARATION**

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: python, hashmap, data-analysis, sql, medium

**MEDIUM | 1Point3Acres | phone-screen | 9 of 15**

# Website Activity Analytics: Unique Users + Session Time

**ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 217/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

**Original:** https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

## QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

**Implement two analytics tasks**

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

**EXAMPLES**

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26
        10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26
        10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:30:00', 'activity': 'logout'},
]
```

## APPROACH

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

## HOW TO THINK ABOUT IT

Unique users per activity is a map from activity to a set of user ids. Time on site needs a definition: pair each login with the next logout of the same user, and say what you do for a login without a logout (here: the session ends at that user's last event). Parse timestamps once, sort each user's events by time, and agree on out-of-order rows and repeated logins before coding.

**Basis:** Written for this question (a grouping and aggregation task; the SQL half of the screen is the same idea with GROUP BY and COUNT(DISTINCT)).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

| TIME | SPACE |
|---|---|
| O(U * N + A * N) | O(U + A) |

**WHY IT IS SLOW**

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

## OPTIMAL SOLUTION · One pass to group, then sessions per user

| TIME | SPACE |
|---|---|
| O(N log N) | O(N) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict
from datetime import datetime

FORMAT = "%Y-%m-%d %H:%M:%S"


def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total


# AVERAGE time on the site per user
def average_time_per_user(records):
```

```
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)


if __name__ == "__main__":
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}

    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900
```

### NOTES FROM CANDIDATES

SQL in the same screen was described as simple and covered joins / aggregation style work, but the exact SQL prompt did not leak.
The interviewer accepted a strong draft when all edge cases were discussed, even though the final code did not cover every case.
Spend time up front on timestamp parsing, ordering, repeated login/logout events, and users with missing terminal actions.

### PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Site tags: array, hashmap, medium

**MEDIUM | 1Point3Acres | onsite-coding | 10 of 15**

# Gold Chain Split into Two Equal Halves

**ASKED 2026-04-21 | 182 DAYS AGO | OLDER**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 201/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

## QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEarrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

## APPROACH

Drill The equal-subset-sum and
partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

**BRUTE FORCE TO SAY OUT LOUD**

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

**OPTIMAL IDEA**

## INTERVIEW-DAY PLAN

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

**DRY-RUN THESE CASES**

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

**LIKELY FOLLOW-UPS**

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

**PITFALLS**

Allowing an empty half. Forgetting the shift of indices after the removed segment.

## HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

**Basis:** Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

## OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

## FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

## FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False


# FOLLOW-UP 1: List every valid (removed segment, cut
# position) pair.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs


if __name__ == "__main__":
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
    assert (1, 1) in valid_pairs([1, 2, 1])
    # both methods agree on many small chains
    import itertools
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))
```

## NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.
Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, O(N²) worst case for enumerating all pairs (output-bound).
Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

**PREPARATION**

Drill The equal-subset-sum and
partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Site tags: array, two-pointer, medium

**MEDIUM | 1Point3Acres | onsite-coding | 11 of 15**

# Sum of Good Subarrays (Adjacent Diff ±1)

**ASKED 2026-02-08 | 254 DAYS AGO | OLDER**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 208/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

## QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

**Input:** integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
**Output:** the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

**EXAMPLES**

```
arr = [3, 5, 6, 7, 6]
```

**Good subarrays**

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

## APPROACH

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

## HOW TO THINK ABOUT IT

A good subarray is a stretch where every step is +1 or every step is -1, so every good subarray ending at index i lies inside the current run. Instead of listing subarrays, keep for each direction the total of all suffix sums inside the run: extending a run of length L by a new element x adds (L + 1) * x to that total. Both directions can be updated in O(1), and the singleton is counted once.

**Basis:** Written for this question (a run-length DP in the style of LeetCode 2262, Total Appeal of A String).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

| TIME | SPACE |
|---|---|
| O(n^2) or O(n^3) | O(1) |

**WHY IT IS SLOW**

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

## OPTIMAL SOLUTION · Two running run totals

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Two running run totals
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total


# BRUTE FORCE (checker)
def good_subarray_sum_brute(a):
    # check every subarray: the oracle for the demo
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            steps = {a[k + 1] - a[k] for k in range(i, j)}
            if steps <= {1} or steps <= {-1}:
                total += sum(a[i:j + 1])
    return total


if __name__ == "__main__":
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
```

```
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4
    # the O(n) answer matches the brute force everywhere
    import itertools
    for n in range(7):
        every = itertools.product([1, 2, 3, 4], repeat=n)
        for a in every:
            assert good_subarray_sum(list(a)) == (
                good_subarray_sum_brute(list(a)))
```

## NOTES FROM CANDIDATES

Walk the array maintaining two run lengths: upRun (current trailing run of +1 steps) and downRun (current trailing run of -1 steps). At index i, only one of them is nonzero (the other resets to 1).

Each index i ends upRun + downRun - 1 good subarrays (subtracting the double-counted singleton). For each ending position, contribute the running prefix-sum tail of the active run.

Multiple candidates report getting stuck — verbalize the "contribution per index" framing out loud before coding.

Easy to mis-handle the reset boundary; double-check with a 2-element example before writing the loop.

## PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.

Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.

Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: data-structure, two-pointer, medium

**MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 12 of 15**

# Run-Length Vector Storage + Dot Product

**ASKED 2026-01-21 | 272 DAYS AGO | OLDER**

**STUDY: TALK** **ALL TIME 5** **30 DAYS 0** **PRIORITY 173/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

**Original:** https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

## QUESTION

**EXAMPLES**

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 =
    9 + 0 + 16 = 25
```

## APPROACH

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask for the encoding format (value, count pairs), whether runs of zeros are stored, the logical length of both vectors (must be equal), and whether updates are needed.

**BRUTE FORCE TO SAY OUT LOUD**

Expand both vectors and multiply element by element. O(n) memory.

**OPTIMAL IDEA**

Two pointers over the run lists. Take the overlap, which is min of the two remaining counts, add value_a * value_b * overlap, subtract the overlap from both, and advance whichever run is exhausted. O(r1 + r2), O(1) extra space.

**DRY-RUN THESE CASES**

The example giving 25. One vector of a single run. Runs that end at the same time. All zeros.

## INTERVIEW-DAY PLAN

**LIKELY FOLLOW-UPS**

A sparse vector against an RLE vector, setting an element, or dot product of k vectors.

**PITFALLS**

Advancing only one pointer when both runs finish together. Overflowing the product for large counts.

## HOW TO THINK ABOUT IT

A run is a stretch where both numbers are constant, so the dot product of two runs is value times value times the length of their overlap. Walk the two run lists like the merge step of merge sort: consume the smaller remaining length from both, and advance whichever run you finished.

**Basis:** LeetCode 1570, Dot Product of Two Sparse Vectors, with run-length encoding as the compression.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

| TIME | SPACE |
|---|---|
| O(logical length) | O(logical length) |

**WHY IT IS SLOW**

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

## OPTIMAL SOLUTION · Two-pointer over runs

| TIME | SPACE |
|---|---|
| O(r1 + r2) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

**FOLLOW-UP 2 How should the data be updated if you set element i?**

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

**FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.**

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_right
from itertools import accumulate


def rle_encode(values):
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1][1] += 1
        else:
            runs.append([v, 1])
    return [tuple(r) for r in runs]


def rle_decode(runs):
    return [v for v, count in runs for _ in range(count)]


# BASE QUESTION (optimal): Two-pointer over runs
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total


# FOLLOW-UP 1, 3: What if one vector is mostly zeros and
# stored sparsely, and the other ...
def sparse_dot_rle(sparse, runs):
    # sparse: {index: value}; runs: RLE of the other
    # vector
    ends = list(accumulate(count for _, count in runs))
    total = 0
    # iterate the small side; binary search the runs
    for index, value in sparse.items():
        k = bisect_right(ends, index)
        if k < len(runs):
            total += value * runs[k][0]
    return total


# FOLLOW-UP 2: How should the data be updated if you set
# element i?
def set_at(runs, index, value):
    # update one position: split a run, then merge
    # neighbours
    flat, pos = [], 0
    for v, count in runs:
        if pos <= index < pos + count:
            flat += [(v, index - pos), (value, 1),
                     (v, pos + count - index - 1)]
        else:
            flat.append((v, count))
        pos += count
    merged = []
    for v, count in flat:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
            merged.append((v, count))
    return merged


if __name__ == "__main__":
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    # 3*1*3 + 0 + 4*2*2 = 25
    assert rle_dot(ra, rb) == 25
    assert rle_dot([], rb) == 0
    dense = sum(x * y for x, y in zip(A, B))
    assert dense == 25
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}
    assert sparse_dot_rle(sparse, rb) == 2 * 1 + 5 * 2
    # setting a value splits and merges runs
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1),
                                (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]
```

**NOTES FROM CANDIDATES**

Two-pointer walk over the two RLE encodings: at each step compute the overlap length min(remainA, remainB), multiply values, advance the pointer whose run is exhausted.
Mention compression-friendly variations: sparse format wins when most entries are 0 (just enumerate non-zero indices and skip zeros entirely in the dot product).
Interviewer cared more about clean data-structure choice and the walk over both encodings than about absolute optimality.

**PREPARATION**

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: hashmap, two-pointer, streaming, medium

**MEDIUM | 1Point3Acres | onsite-coding | 13 of 15**

# Two-Day Log Intersection under Memory Limit

**ASKED 2025-11-27 | 327 DAYS AGO | OLDER**

**STUDY: TYPE** **ALL TIME 50.4** **30 DAYS 46.9** **PRIORITY 24/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

**Original:** https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

## QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

**Input:** two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

**Follow-up 1:** complexity of set-intersection on the file contents.
**Follow-up 2:** memory cannot hold either file fully — design a streaming solution.

**EXAMPLES**

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients
    {c1, c2}.
Output: [X]
```

## APPROACH

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

## HOW TO THINK ABOUT IT

In memory it is a hash map from object to the set of clients, filtered to objects that appear in both files and have at least two clients. When the files do not fit in memory, sort both by object id (external sort) and walk them together: all rows of one object are adjacent, so you only ever hold the clients of one object at a time.

**Basis:** LeetCode 349, Intersection of Two Arrays, with a distinct-client condition and a limit on memory (external sort and a two-pointer merge).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

| TIME | SPACE |
|---|---|
| O(n * m) | O(n + m) |

**WHY IT IS SLOW**

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

## OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

| TIME | SPACE |
|---|---|
| O(n + m) | O(n + m) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)


def groups(rows):
    # rows sorted by object: yield (object, its clients)
    current, clients = None, set()
    for _, obj, client in rows:
        if obj != current:
            if current is not None:
                yield current, clients
            current, clients = obj, set()
        clients.add(client)
    if current is not None:
        yield current, clients


# FOLLOW-UP 1: Why prefer sort and two pointers to a
# streaming hash map?
def log_objects_sorted(rows1, rows2):
    # memory limit: both files arrive sorted by object (an
    # external sort), so only one object's clients are
    # held
    out = []
    g1, g2 = groups(rows1), groups(rows2)
    a, b = next(g1, None), next(g2, None)
    while a and b:
        if a[0] == b[0]:
            # same object in both files
            if len(a[1] | b[1]) >= 2:
                out.append(a[0])
            a, b = next(g1, None), next(g2, None)
        elif a[0] < b[0]:
            a = next(g1, None)
        else:
            b = next(g2, None)
    return out


if __name__ == "__main__":
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    assert log_objects(day1, day2) == ["X"]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])

    assert log_objects_sorted(by_object(day1),
                              by_object(day2)) == ["X"]
    # one client in total is not enough
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []
    assert log_objects_sorted(one, one) == []
    # clients may be split across the two days
    d1, d2 = [(1, "A", "c1")], [(2, "A", "c2")]
    assert log_objects(d1, d2) == ["A"]
    assert log_objects_sorted(d1, d2) == ["A"]
    assert log_objects([], day2) == []
```

## NOTES FROM CANDIDATES

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

## PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, grid, medium

**MEDIUM | 1Point3Acres | onsite-coding | 14 of 15**

# Rotate Any m×n 2D Matrix

**STUDY: TYPE** **ALL TIME 56.3** **30 DAYS 0** **PRIORITY 57/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #48 Rotate Image

**LeetCode match:** LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

## QUESTION

**EXAMPLES**

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
    [[4,1],[5,2],[6,3]] (3×2).
```

## APPROACH

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

**BRUTE FORCE TO SAY OUT LOUD**

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

**OPTIMAL IDEA**

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

**DRY-RUN THESE CASES**

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

**LIKELY FOLLOW-UPS**

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

**PITFALLS**

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

## HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

**Basis:** Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

| TIME | SPACE |
|---|---|
| O(m * n) | O(m * n) |

**WHY IT IS SLOW**

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

## OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

| TIME | SPACE |
|---|---|
| O(mn) | O(mn) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

## FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

### FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# FOLLOW-UP 2, 3: How would you rotate by k * 90 degrees?
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


if __name__ == "__main__":
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
```

```
    assert rotate_cw([]) == []
```

### NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing.
Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

### PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

**STUDY HERE: topic 1, Arrays, Hashing and Two Pointers**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: hashmap, simulation, easy

**EASY | 1Point3Acres | onsite-coding | 15 of 15**

# Robot Status Message Deduplication Window

**ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS**

**STUDY: TYPE** **ALL TIME 51.2** **30 DAYS 33** **PRIORITY 34/228**

## ABOUT THIS QUESTION

**Topic:** Arrays, Hashing and Two Pointers

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #359 Logger Rate Limiter

**LeetCode match:** LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

## QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

**Logger API variant**

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

**EXAMPLES**

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

## APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

## HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

**Basis:** LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

| TIME | SPACE |
|---|---|
| **O(displayed)** per event | **O(displayed)** |

**WHY IT IS SLOW**

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

## OPTIMAL SOLUTION · Hash map of last displayed time

| TIME | SPACE |
|---|---|
| **O(1)** per event | **O(distinct messages)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_left, bisect_right, insort


# BASE QUESTION (optimal): Hash map of last displayed time
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True
```

```
if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

## NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

## PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.